

RESULTS

HelixLLM Phase-4 Vision (VLM) Boot Harness — END-TO-END PROOF (§11.4.108)

Status	COMPLETE — ALL-GREEN. Real Qwen2.5-VL-3B multimodal server booted via the containers submodule, RED/GREEN §11.4.115 polarity pair both PASS (each re-run twice), coder untouched throughout.
Run-id	helixllm_vision_boot_20260707T215007Z
Date	2026-07-07 (UTC), host x86_64, RTX 5090 (32607 MiB total VRAM), rootless podman
Branch / Track	feature/helixllm-full-extension
Component	submodules/helix_llm/cmd/visiongen-boot/ (admit-check / boot / down / status)
Sibling lane	docs/qa/helixllm_imagegen_* / helixllm_videogen_* (image-gen / video-gen GPU BURST lanes) — vision is the WARM co-resident lane (vrambroker.ClassVLM), stays up after boot instead of tearing down.

1. What was booted, and how (§11.4.76 / §11.4.161)

- Booted **through the containers-submodule compose.Orchestrator** — NOT ad-hoc podman/docker commands. Rootless podman, NVIDIA CDI GPU device injection (nvidia.com/gpu=all).
- Image localhost/helixllm/llamacpp-router:cuda12.8-sm120 — the ALREADY-BUILT router image (no rebuild, §11.4.74 reuse-don't-reimplement), run as a multimodal llama-server (GGUF weights + libmtmd mmproj projector).
- Model: Qwen2.5-VL-3B-Instruct-Q4_K_M.gguf + mmproj-Qwen2.5-VL-3B-Instruct-Q8_0.gguf, env-injected model directory (§CONST-045/ §11.4.28 — no hardcoded host path in compose.vision.yml itself).

- Own port :18439, distinct from the live coder (:18434) and every sibling capability lane (:18435–:18443) per §11.4.119 single-resource-owner.

2. Admission gate (broker-first, fail-closed §11.4.6)

admit_check.txt:

```
VRAM budget (nvidia-smi): total=32607MiB used=19436MiB free=12685MiB need=
ADMIT-OK: VLM footprint admitted co-resident (coder stays live) – warm tie
admit-check exit=0
```

The VLM footprint (5 GiB placeholder, phase3-measured ~4.1 GiB actual peak + margin) was admitted against MEASURED free VRAM (12685 MiB free at the time) BEFORE any container boot — never a raw VRAM grab.

3. Root-cause-fix caught mid-run (§11.4.102)

The FIRST boot attempt (boot.txt) admitted fine, brought the container UP, but timed out on /health after 100 polls (5 minutes):

```
UP-OK: helixllm_visiongen visiongen via containers submodule orchestrator
HEALTH-TIMEOUT: visiongen /health did not answer within 5m0s (100 polls)
BLOCKED: visiongen service never became healthy on :18439
exit status 4
```

visiongen_container_log_1.txt captured the actual failure inside the container: error: invalid argument: llama-server — the router image's ENTRYPOINT is already ["llama-server"] (confirmed via router_image_entrypoint.txt, podman inspect .Config.Entrypoint), so the compose command: list must NOT re-add llama-server as its own first argv token — doing so duplicates the binary name as argv[1] and llama-server rejects it. compose.vision.yml was corrected to drop that leading token (documented in the file's own header comment as a permanent guard against regression), and the retry succeeded:

boot_retry1.txt:

```
VRAM budget (nvidia-smi): total=32607MiB used=19436MiB free=12685MiB need=
ADMIT-OK: VLM footprint admitted co-resident (coder stays live) – warm tie
UP-OK: helixllm_visiongen visiongen via containers submodule orchestrator
HEALTH-OK: visiongen /health after 2 polls (status=200)
BOOT-HEALTH-OK: visiongen /health answered. VLM stays UP (warm tier, coder
Run `visiongen-boot down <compose-file> <project>` for single-owner teardo
boot exit=0
```

4. §11.4.115 RED/GREEN polarity pair — both re-run twice

The single test source `vision_boot_test.sh` toggles on `RED_MODE`:

RED (`RED_MODE=1`, reproduce-the-defect on the torn-down artifact):

```
test_red_mode1.txt:          PASS: defect reproduced: :18439 unreachable aft
test_red_mode1_rerun2.txt: PASS: defect reproduced: :18439 unreachable aft
```

GREEN (`RED_MODE=0`, standing regression guard — boot + REAL multimodal inference):

```
test_green_mode0.txt:          PASS: vision endpoint served a real multimoda
                                (coder untouched, was up before, Up after):
                                "A red square is displayed on a black backgr
test_green_mode0_rerun2.txt: PASS: vision endpoint served a real multimoda
                                (coder untouched, was up before, Up after):
                                "The image shows a red square on a black bac
```

The two GREEN runs produced **two different sentences** describing the SAME generated test image (a 128×128 black PNG with a red rectangle drawn at 10,10–60,60 via ImageMagick, `test_image.png`) — direct evidence this is a live model sampling a real answer each call, not a cached/canned string (§11.4.2/§11.4.5 anti-bluff).

5. Raw inference evidence (§11.4.69 sink-side, not health-200-only)

`inference_response.json` (captured during a manual confirmation call, same harness/image/model):

```
{"choices":[{"finish_reason":"stop","index":0,"message":
    {"role":"assistant",
"content":"The image shows a red square and a green circle on a
    black background."}],
"model":"/models/Qwen2.5-VL-3B-Instruct-Q4_K_M.gguf",
"system_fingerprint":"b1-3b4fca1",
"usage":{"completion_tokens":16,"prompt_tokens":109,"total_tokens":125},
"timings":
    {"prompt_ms":109.965,"predicted_ms":43.118,"predicted_per_second":
```

Real llama.cpp server timing/token telemetry, real model path on disk, real `system_fingerprint` — this is a genuine multimodal completion, not a metadata-only or config-only pass.

6. Coder untouched throughout (§11.4.122 single-owner)

```
coder_before.txt:      helixllm-coder Up 32 hours 8080/tcp, 50052/tcp
coder_after_final.txt: helixllm-coder Up 32 hours 8080/tcp, 50052/tcp
                      helixllm_visiongen_visiongen_1 Up 9 seconds 0.0.0.
```

The coder container’s uptime (32 hours) is IDENTICAL before and after every boot/teardown/inference cycle in this run — it was never stopped, restarted, or paused by this harness, confirming the “this harness DOES NOT pause the live coder autonomously” design invariant (main.go header comment).

7. Evidence manifest

File	What it proves
admit_check.txt	Broker VRAM admission gate, fail-closed
boot.txt	First boot attempt — health-timeout (pre-fix)
visiongen_container_log_1.txt	Root cause: duplicated llama-server argv token
router_image_entrypoint.txt	podman inspect confirming the image’s own ENTRYPOINT
boot_retry1.txt	Post-fix boot — healthy after 2 polls
test_red_mode1.txt / _rerun2	RED — defect reproduced on torn-down artifact (x2)
test_green_mode0.txt / _rerun2	GREEN — real inference, non-identical real completions (x2)
inference_request.json / inference_response.json	Raw multimodal request/response with real llama.cpp telemetry
test_image.png	The generated test fixture (red rectangle on black)
coder_before.txt / coder_after_final.txt	Coder uptime unchanged — single-owner cleanup respected
podman_ps_after_boot.txt / podman_ps_after_boot_timeout.txt	Raw container state snapshots at each phase
inference_curl_stderr.txt	curl transport diagnostics for the inference call

File	What it proves
magick.log	ImageMagick test-fixture generation log (empty = clean exit)

8. Honest boundary (§11.4.6)

This proof covers the boot/admit/teardown/inference lifecycle of the DEFAULT-provisioned 3B model on ONE host/GPU. It does NOT prove: the 7B model variant (documented env-override path, not exercised here), behaviour under concurrent burst-lane (image/video) contention (§11.4.119 partitioning is enforced by the broker's `ErrBurstInUse` path but not exercised in this run), or multi-request concurrent-load characteristics (§11.4.85 stress — tracked as a separate follow-up, not claimed here).